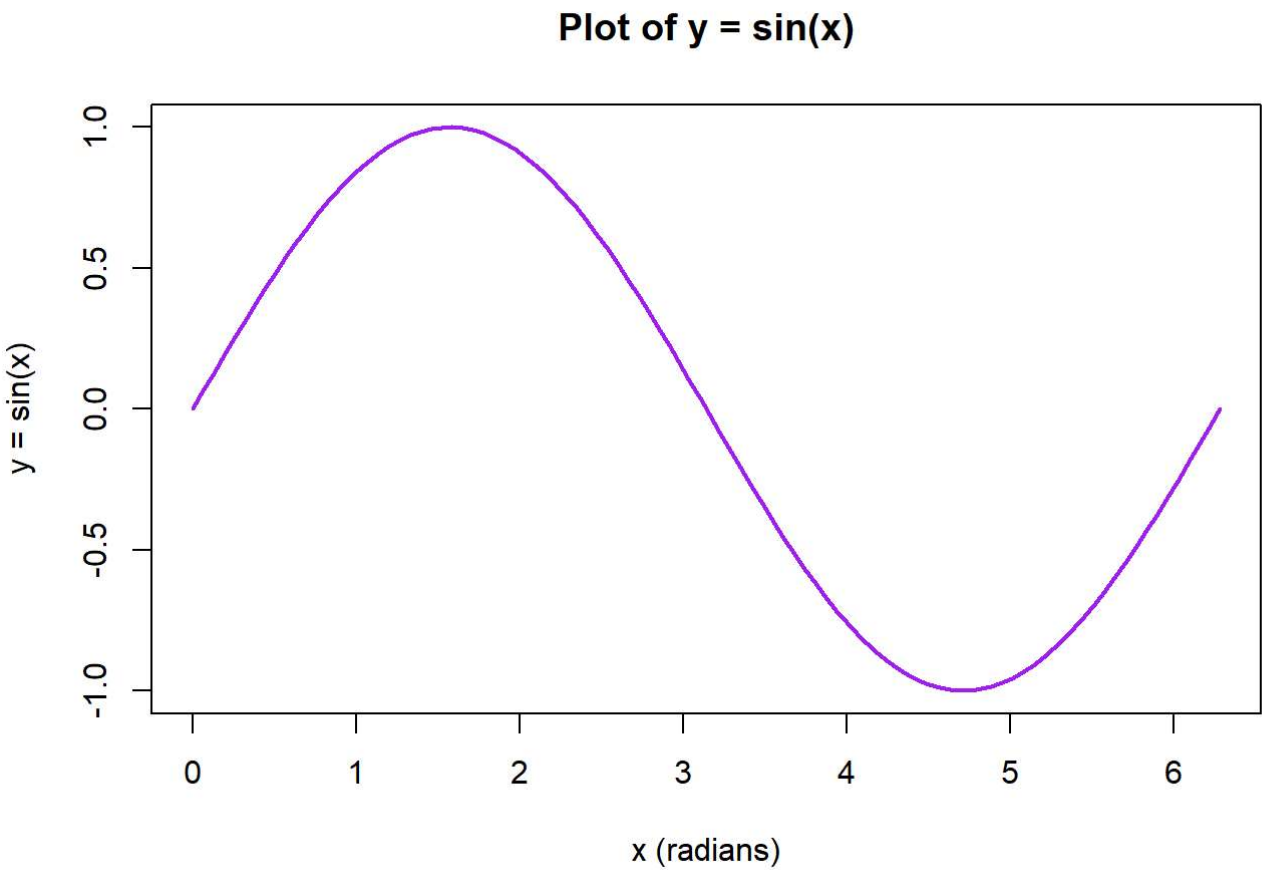
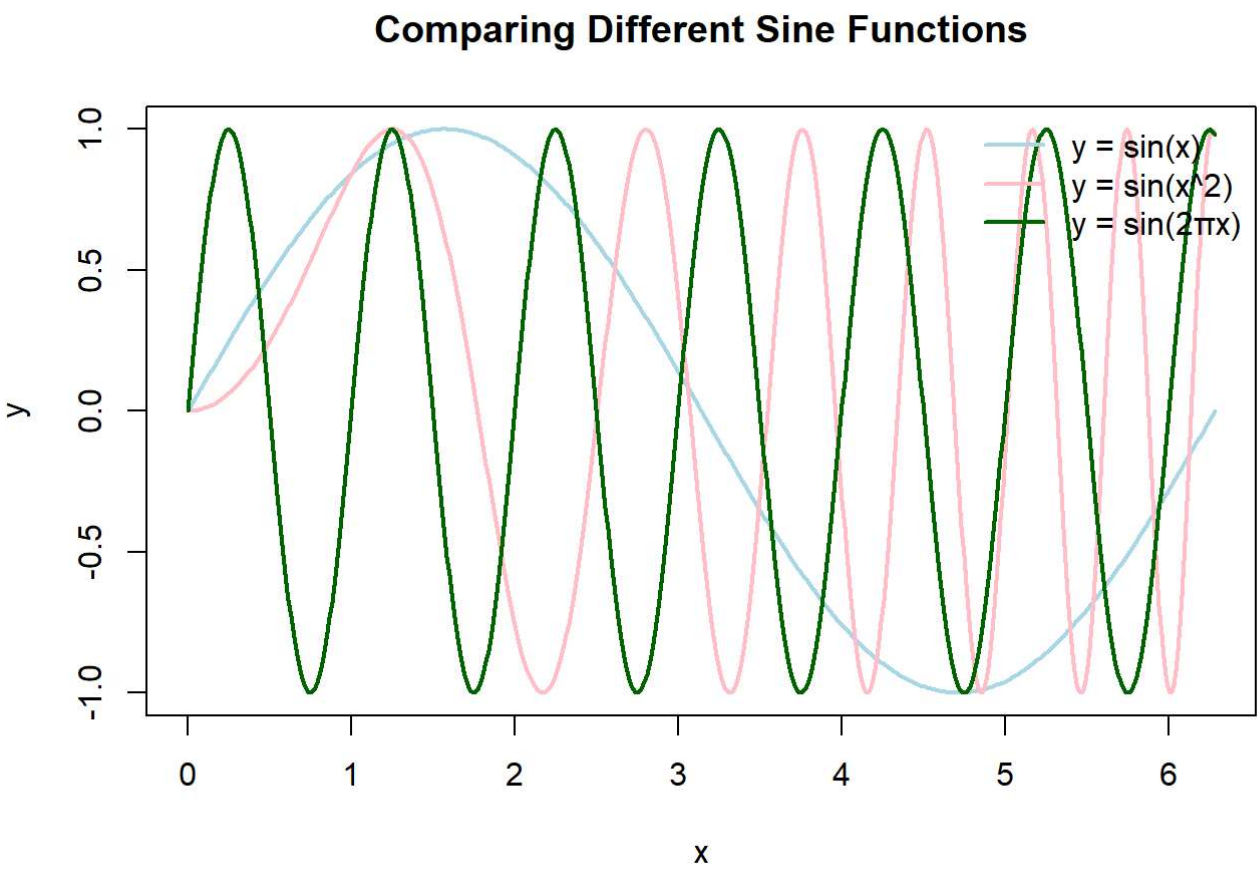
R code IDAS assignment

2025-09-03

```
#graph  1
# x values
x <- seq(0, 2*pi, length.out = 100)

# Compute y = sin(x)
y <- sin(x)

# Plotting the function
plot(x, y, type = "l", col = "purple", lwd = 2,
     xlab = "x (radians)",
     ylab = "y = sin(x)",
     main = "Plot of y = sin(x)")
```



```r
#graph 2
# x values
x <- seq(0, 2*pi, length.out = 500)

# Computing functions
y1 <- sin(x)
y2 <- sin(x^2)
y3 <- sin(2*pi*x)

# Plotting the first function
plot(x, y1, type = "l", col = "lightblue", lwd = 2,
     xlab = "x", ylab = "y",
     main = "Comparing Different Sine Functions")

# Adding the other two functions
lines(x, y2, col = "pink", lwd = 2)
lines(x, y3, col = "darkgreen", lwd = 2)

# Adding legend inside the figure
legend("topright",
       legend = c("y = sin(x)", "y = sin(x^2)", "y = sin(2πx)"),
       col = c("lightblue", "pink", "darkgreen"),
       lwd = 2, bty = "n")
```

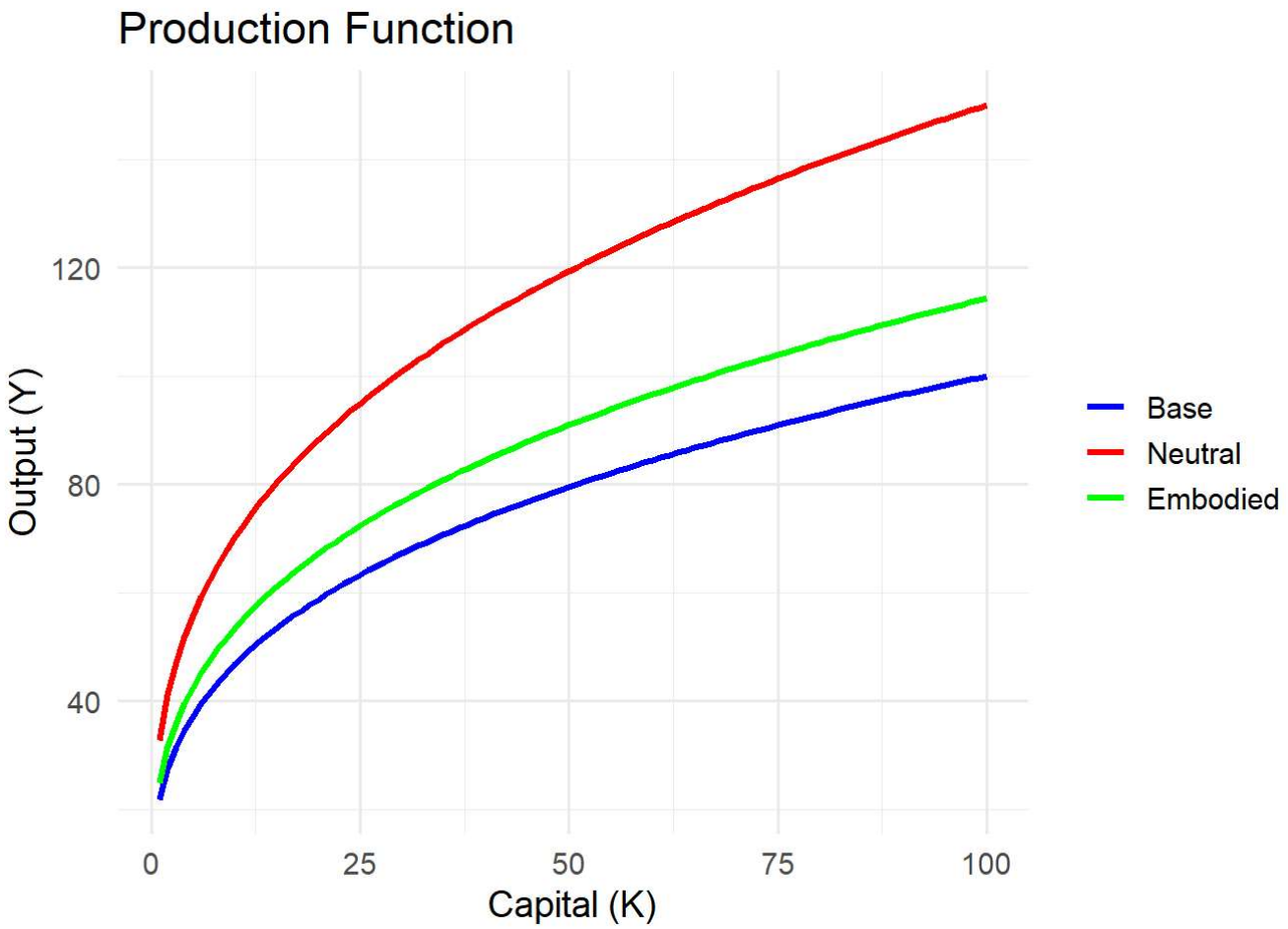



```r
#graph 3
library(ggplot2)
library(reshape2)
```

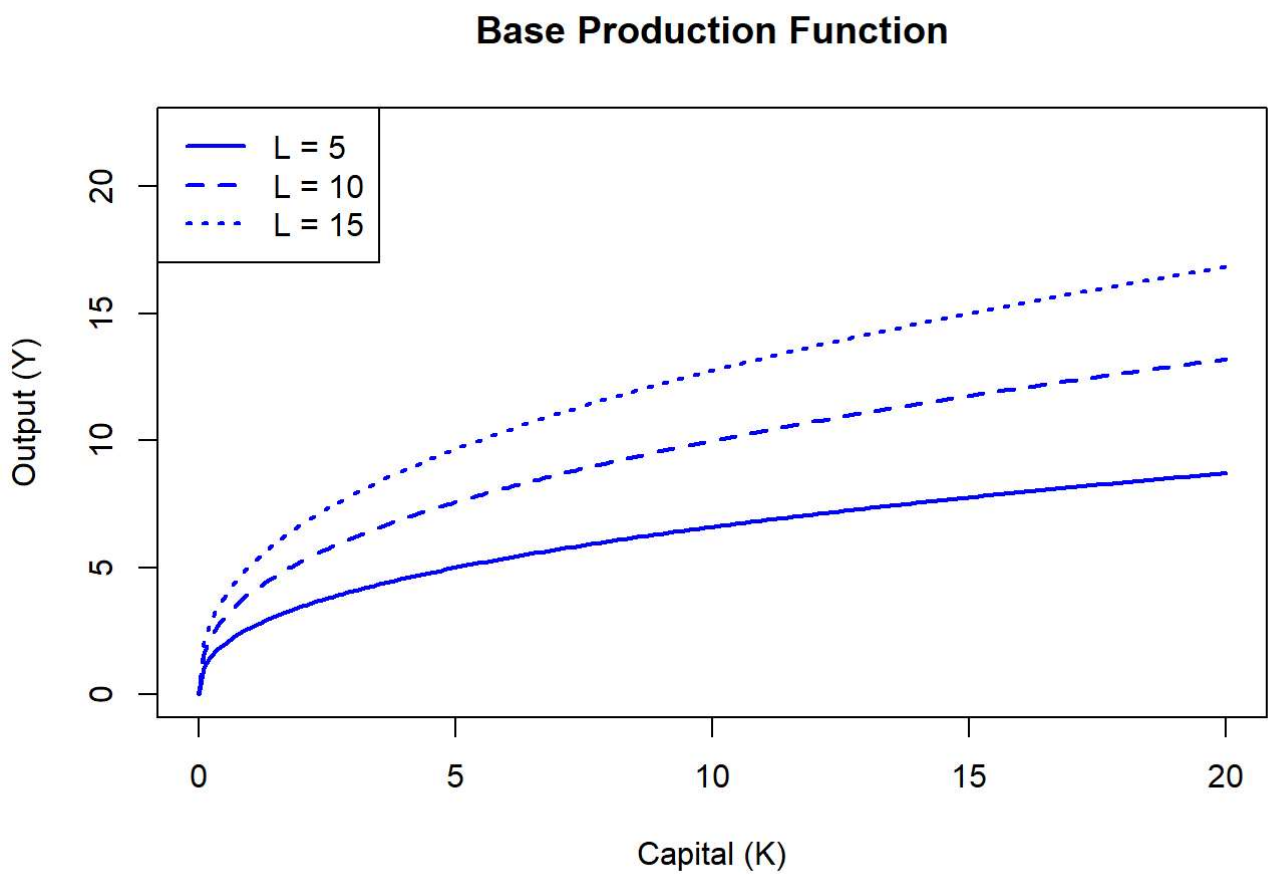
```
## Warning: package 'reshape2' was built under R version 4.3.3
```

```r
# Parameters
alpha <- 0.33
Lfix  <- 100
A     <- 1
A_new <- 1.5
Kmax  <- 100

# Production functions
fY <- function(K, L, A = 1, alpha = 0.33) A * K^alpha * L^(1 - alpha)
fY_embodied <- function(K, L, A = 1, alpha = 0.33) (A * K)^alpha * L^(1 - alpha)

# Data for plotting
df <- data.frame(K = seq(1, Kmax, by = 1))
df$Base      <- fY(df$K, Lfix, A = 1, alpha = alpha)
df$Neutral   <- fY(df$K, Lfix, A = A_new, alpha = alpha)
df$Embodied  <- fY_embodied(df$K, Lfix, A = A_new, alpha = alpha)

# Reshape to long format
df_long <- melt(df, id.vars = "K",
                variable.name = "Case", value.name = "Y")

# Plot with custom colors
ggplot(df_long, aes(x = K, y = Y, color = Case)) +
  geom_line(size = 1.2) +
  scale_color_manual(values = c("Base" = "blue",
                                "Neutral" = "red",
                                "Embodied" = "green")) +
  labs(title = "Production Function",
       x = "Capital (K)", y = "Output (Y)") +
  theme_minimal(base_size = 14) +
  theme(legend.title = element_blank())
```

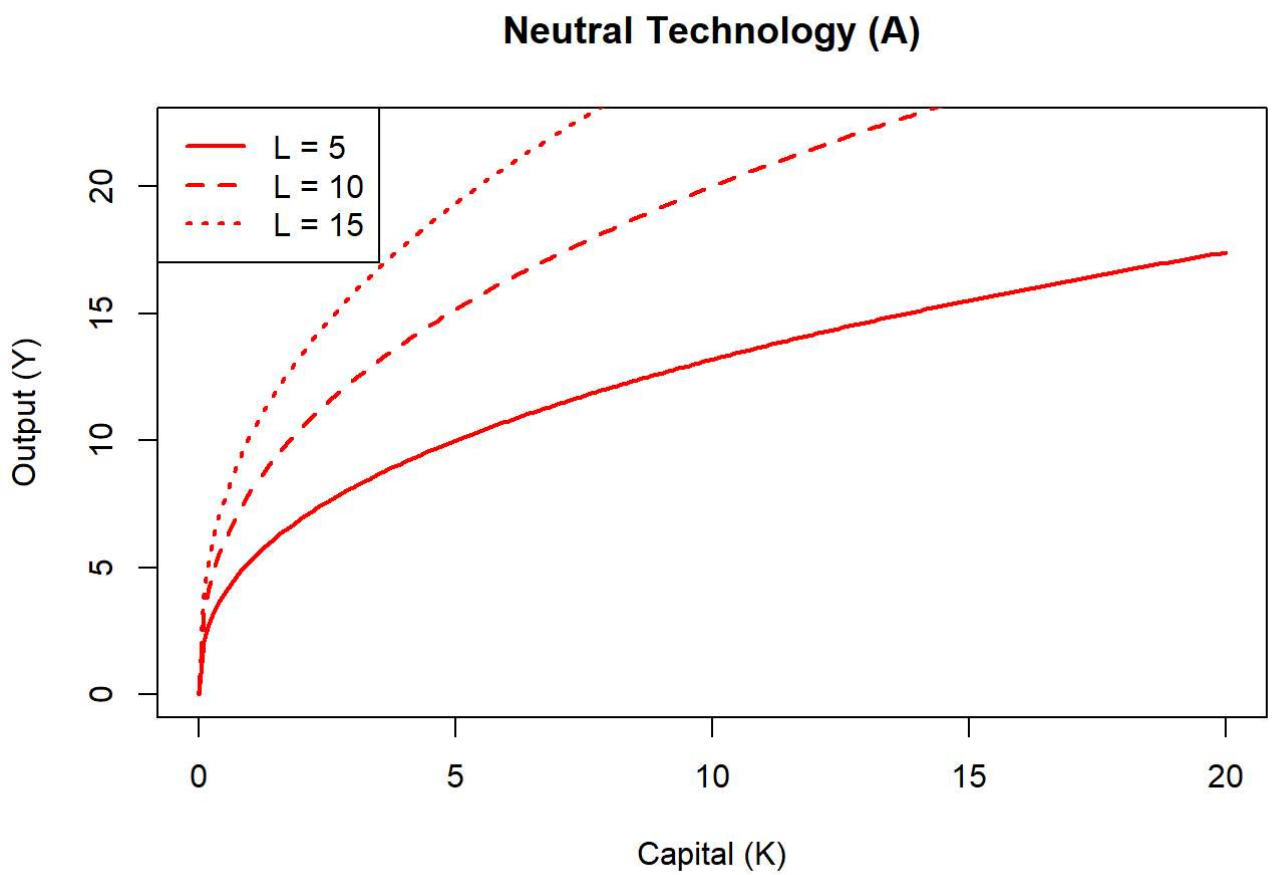
```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ℹ Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



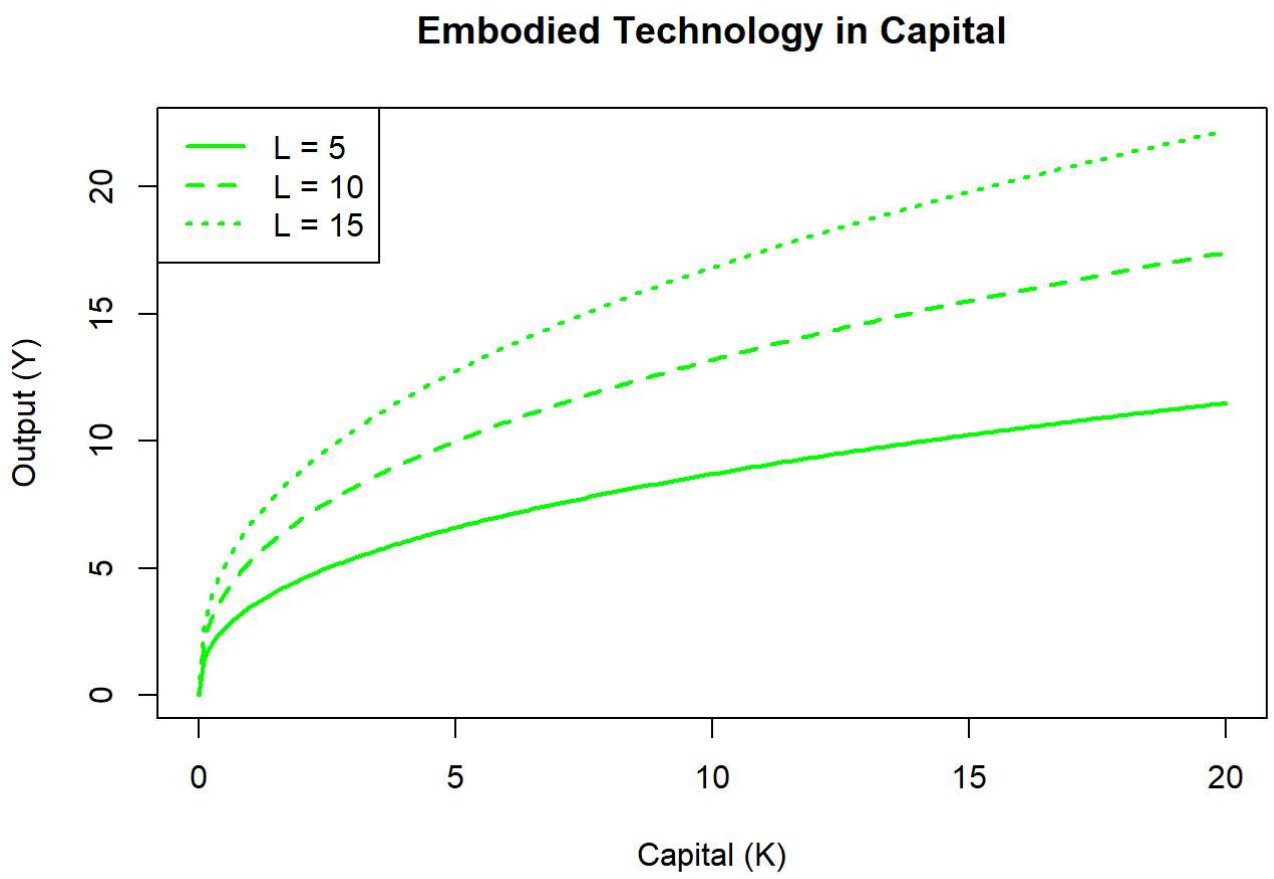

```
##graph 4
# Parameters
alpha <- 0.4
A <- 2
K_vals <- seq(0, 20, length.out = 200)

# Fix a few L values to compare
L_vals <- c(5, 10, 15)

# Base Production Function
plot(K_vals, K_vals^alpha * L_vals[1]^(1-alpha), type="l", lwd=2, col="blue",
     ylim=c(0, max((A*K_vals)^alpha * max(L_vals)^(1-alpha))),
     xlab="Capital (K)", ylab="Output (Y)", main="Base Production Function")
for (L in L_vals[-1]) {
  lines(K_vals, K_vals^alpha * L^(1-alpha), lwd=2, col="blue", lty=L/5)
}
legend("topleft", legend=paste("L =", L_vals), col="blue", lty=1:length(L_vals), lwd=2)
```



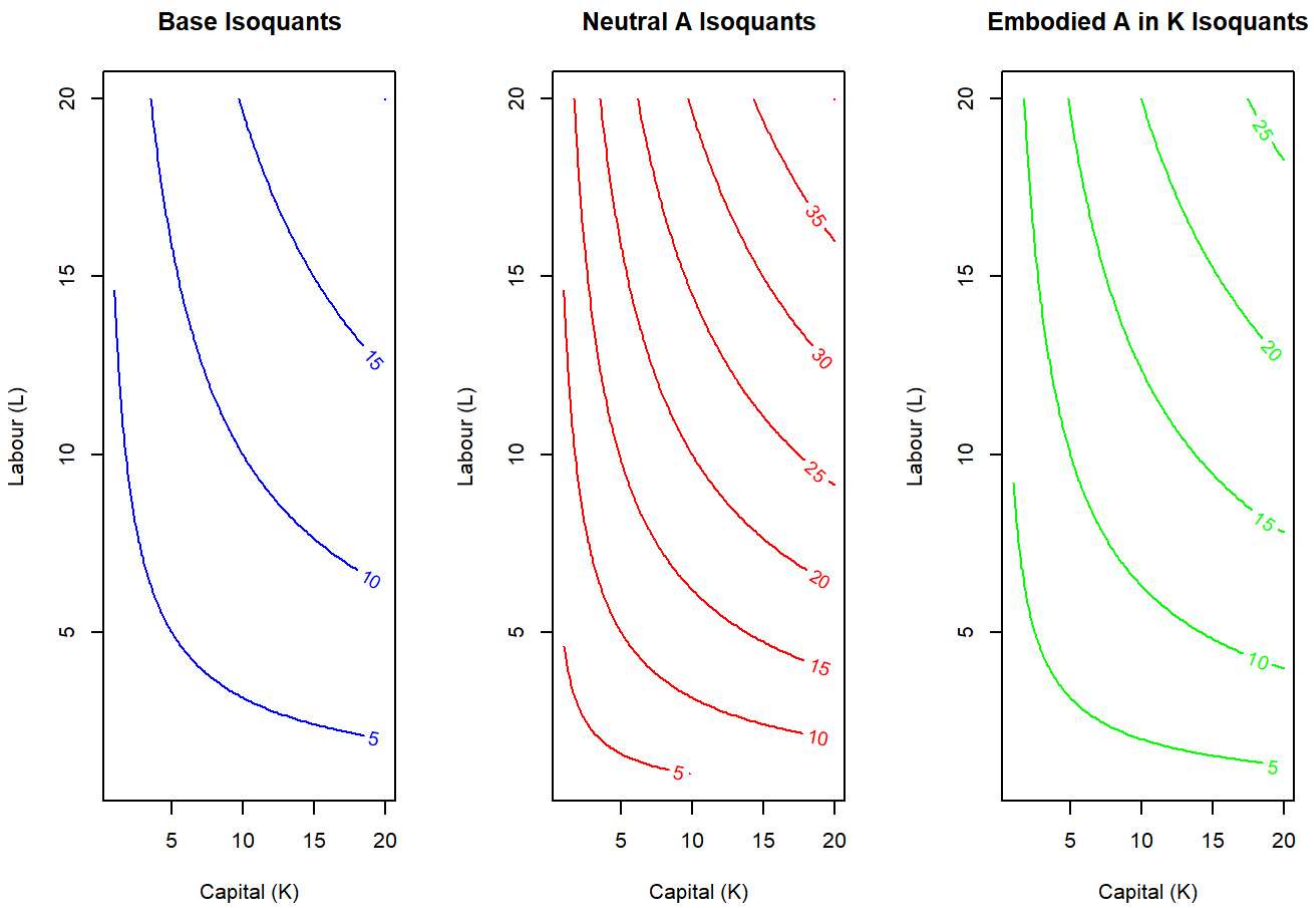

```
# Neutral Technology
plot(K_vals, A*K_vals^alpha * L_vals[1]^(1-alpha), type="l", lwd=2, col="red",
     ylim=c(0, max((A*K_vals)^alpha * max(L_vals)^(1-alpha))),
     xlab="Capital (K)", ylab="Output (Y)", main="Neutral Technology (A)")
for (L in L_vals[-1]) {
  lines(K_vals, A*K_vals^alpha * L^(1-alpha), lwd=2, col="red", lty=L/5)
}
legend("topleft", legend=paste("L =", L_vals), col="red", lty=1:length(L_vals), lwd=2)
```



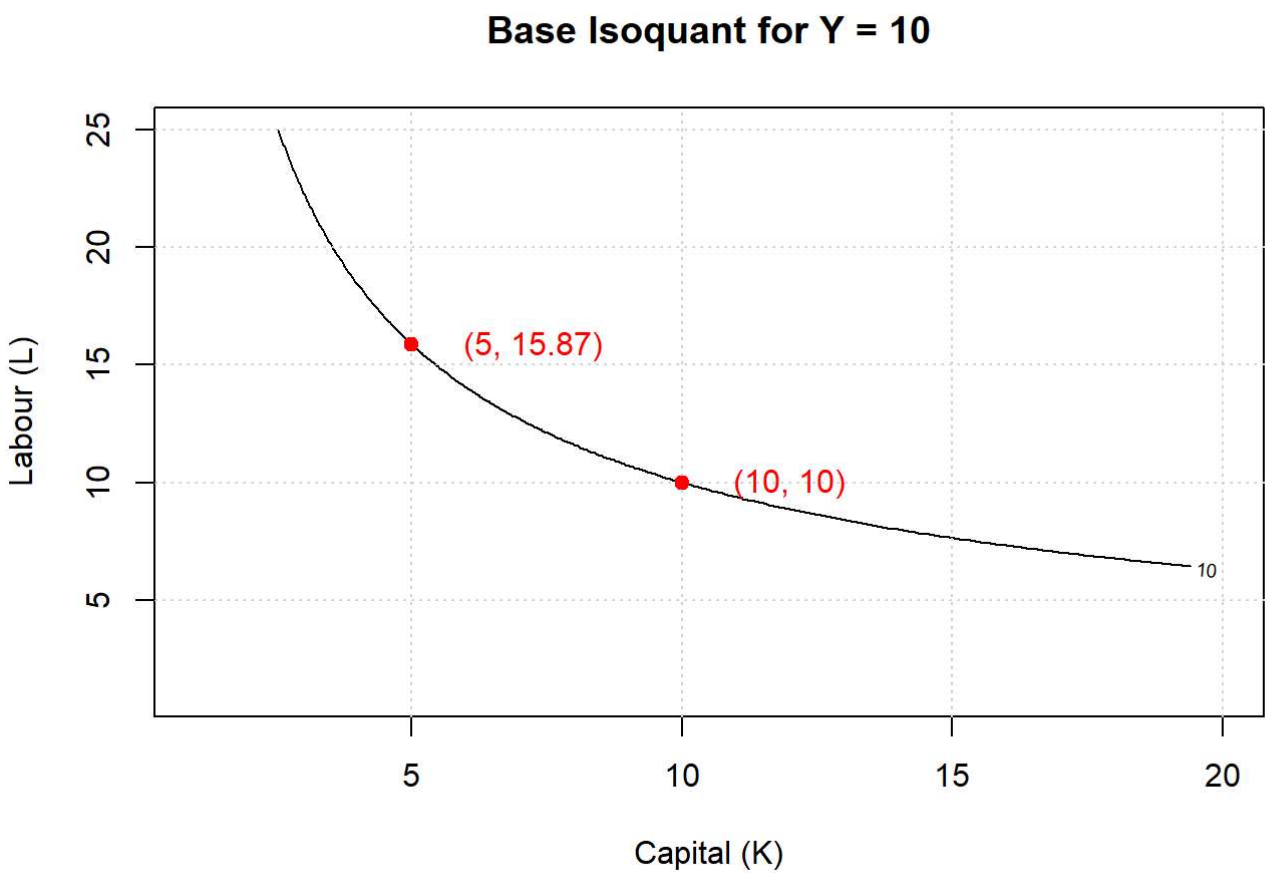

```
# Embodied Technology in Capital
plot(K_vals, (A*K_vals)^alpha * L_vals[1]^(1-alpha), type="l", lwd=2, col="green",
     ylim=c(0, max((A*K_vals)^alpha * max(L_vals)^(1-alpha))),
     xlab="Capital (K)", ylab="Output (Y)", main="Embodied Technology in Capital")
for (L in L_vals[-1]) {
  lines(K_vals, (A*K_vals)^alpha * L^(1-alpha), lwd=2, col="green", lty=L/5)
}
legend("topleft", legend=paste("L =", L_vals), col="green", lty=1:length(L_vals), lwd=2)
```


Embodied Technology in Capital
L = 5
L = 10
L = 15
Output (Y)
0
5
10
15
20
0
5
10
15
20
Capital (K)

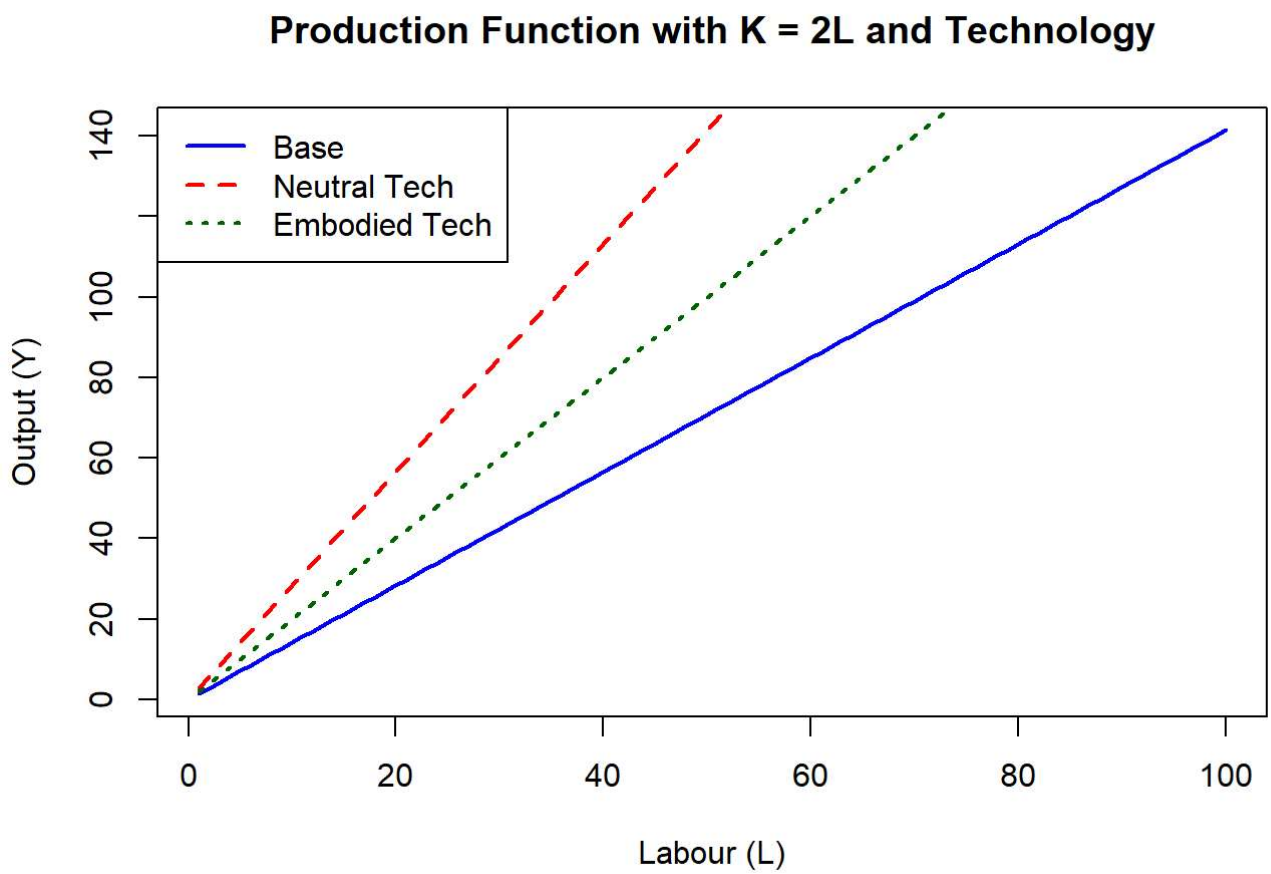
```r
#GRAPH 5
# Parameters
alpha <- 0.4
A <- 2

# Grid of K and L
K_vals <- seq(1, 20, length.out = 100)
L_vals <- seq(1, 20, length.out = 100)
grid <- expand.grid(K = K_vals, L = L_vals)

# Functions
grid$Y_base     <- grid$K^alpha * grid$L^(1-alpha)
grid$Y_neutral  <- A * grid$K^alpha * grid$L^(1-alpha)
grid$Y_embodied <- (A*grid$K)^alpha * grid$L^(1-alpha)

# Reshape to matrices
Y_base_mat     <- matrix(grid$Y_base, nrow=length(K_vals))
Y_neutral_mat  <- matrix(grid$Y_neutral, nrow=length(K_vals))
Y_embodied_mat <- matrix(grid$Y_embodied, nrow=length(K_vals))

# Plot isoquants
par(mfrow=c(1,3))

contour(K_vals, L_vals, Y_base_mat, nlevels=6, col="blue",
        xlab="Capital (K)", ylab="Labour (L)", main="Base Isoquants")

contour(K_vals, L_vals, Y_neutral_mat, nlevels=6, col="red",
        xlab="Capital (K)", ylab="Labour (L)", main="Neutral A Isoquants")

contour(K_vals, L_vals, Y_embodied_mat, nlevels=6, col="green",
        xlab="Capital (K)", ylab="Labour (L)", main="Embodied A in K Isoquants")
```



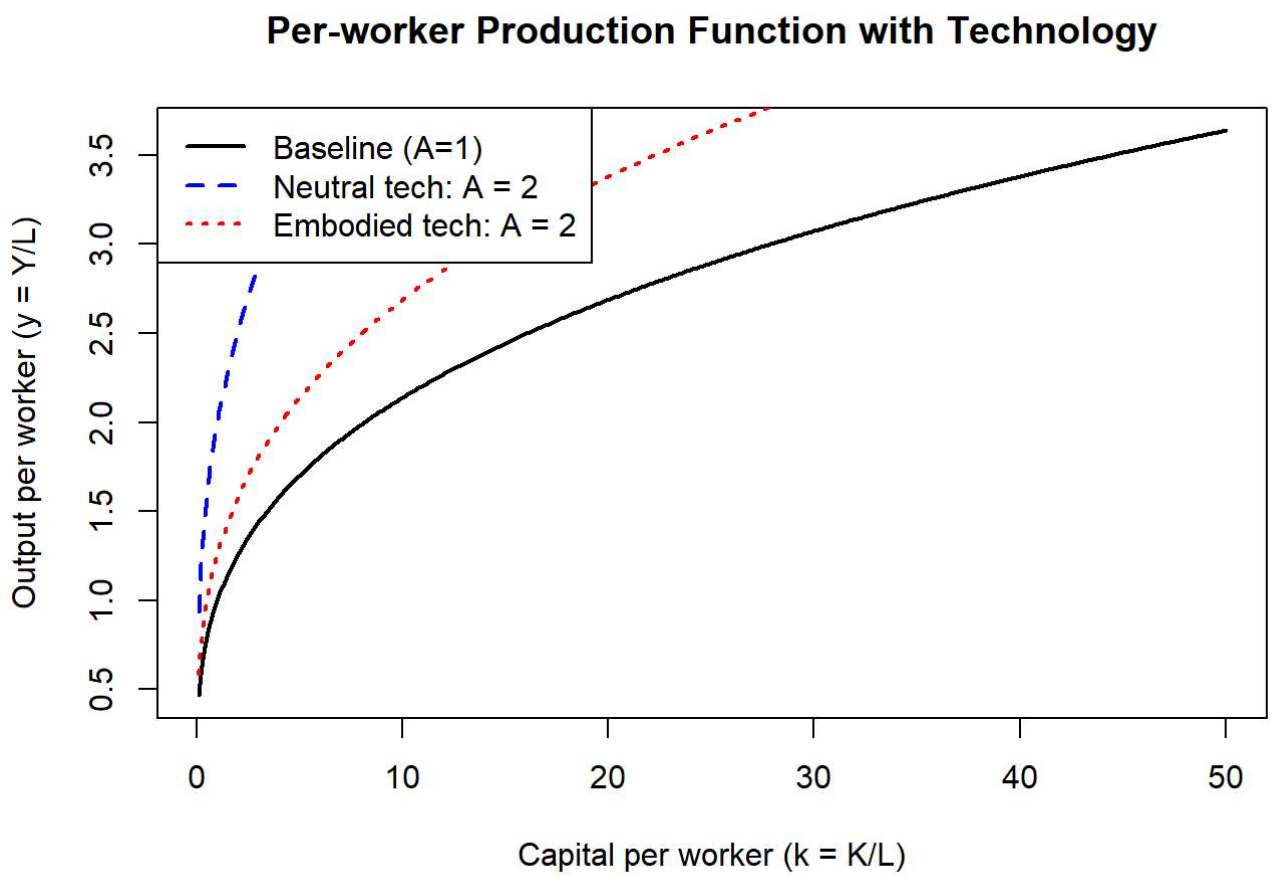

```r
#graph 6
# Parameters
alpha <- 0.4
Y0 <- 10

# Grid of K and L values
K_vals <- seq(1, 20, length.out = 200)
L_vals <- seq(1, 25, length.out = 200)

# Compute output matrix (correct orientation)
Y_base <- outer(K_vals, L_vals,
                function(K, L) K^alpha * L^(1-alpha))

# Plot isoquant for Y0 (no transpose here!)
contour(K_vals, L_vals, Y_base, levels = Y0,
        xlab = "Capital (K)", ylab = "Labour (L)",
        main = paste("Base Isoquant for Y =", Y0))
grid()

# Points on the isoquant
K_points <- c(5, 10)
L_points <- (Y0 / (K_points^alpha))^(1/(1-alpha))

points(K_points, L_points, pch = 19, col = "red")
text(K_points + 0.6, L_points,
     labels = paste0("(", K_points, ", ", round(L_points, 2), ")"),
     pos = 4, col = "red")
```



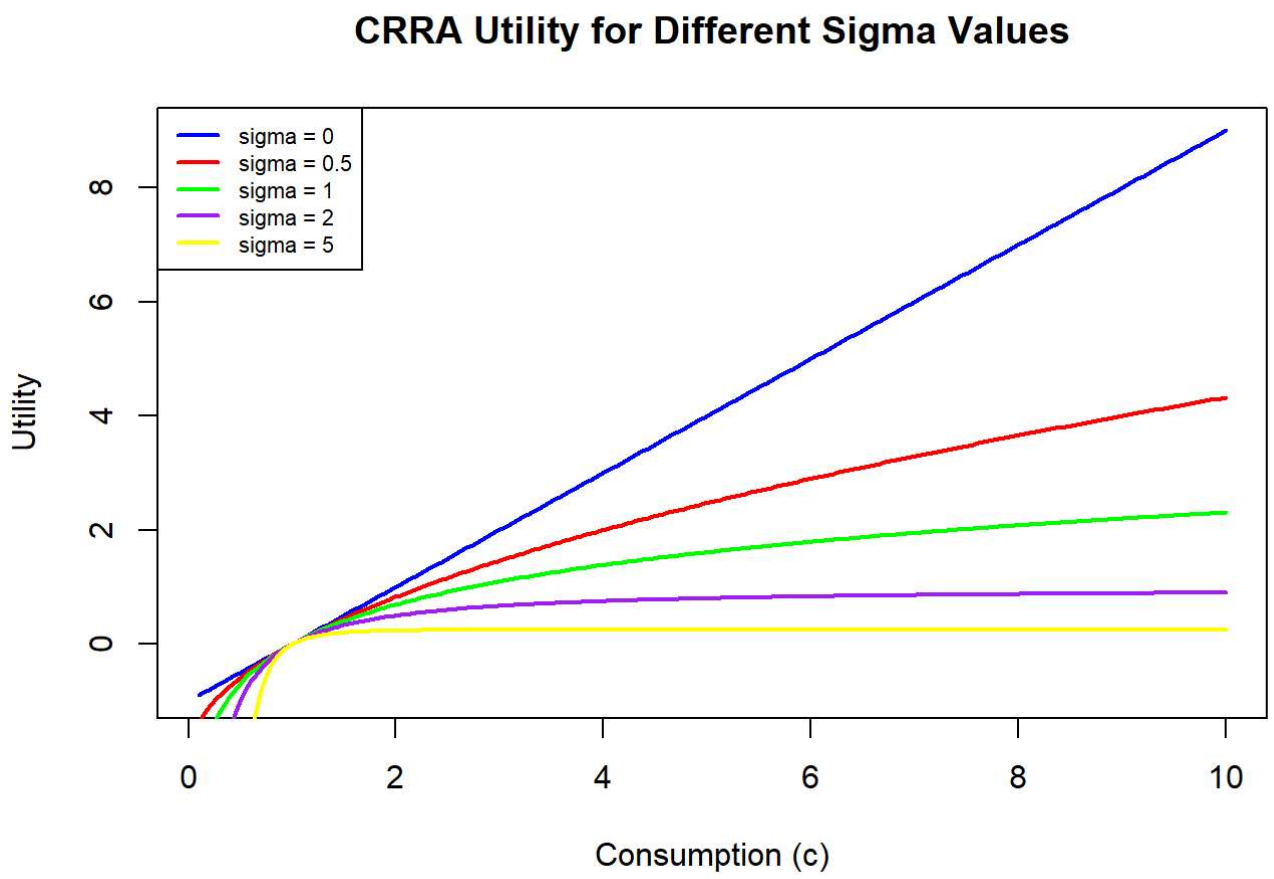

```
#graph 7
# Parameters
alpha <- 0.5
A <- 2                  # technology factor
L <- 1:100              # Labour
K <- 2 * L              # Capital = 2L

# Base production function
Y_base <- K^alpha * L^(1 - alpha)

# Neutral technology (A multiplies output)
Y_neutral <- A * Y_base

# Embodied technology (A multiplies capital only)
Y_embodied <- (A * K)^alpha * L^(1 - alpha)

# Plot
plot(L, Y_base, type = "l", lwd = 2, col = "blue",
     xlab = "Labour (L)", ylab = "Output (Y)",
     main = "Production Function with K = 2L and Technology")
lines(L, Y_neutral, col = "red", lwd = 2, lty = 2)
lines(L, Y_embodied, col = "darkgreen", lwd = 2, lty = 3)

# Legend
legend("topleft", legend = c("Base", "Neutral Tech", "Embodied Tech"),
       col = c("blue", "red", "darkgreen"), lty = c(1, 2, 3), lwd = 2)
```




```
#Graph 8
# Cobb-Douglas per-worker production function
# Parameters
alpha <- 0.33
A <- 2     # technology level

# Capital-labour ratio (k = K/L)
k <- seq(0.1, 50, length.out = 500)

# Per-worker production functions
y_baseline <- k^alpha                    # no technology
y_neutral  <- A * k^alpha                # neutral technology
y_embodied <- (A^alpha) * k^alpha        # technology embodied in capital

# Plot
plot(k, y_baseline, type = "l", lwd = 2,
     xlab = "Capital per worker (k = K/L)",
     ylab = "Output per worker (y = Y/L)",
     main = "Per-worker Production Function with Technology")
lines(k, y_neutral, col = "blue", lwd = 2, lty = 2)
lines(k, y_embodied, col = "red", lwd = 2, lty = 3)

legend("topleft",
       legend = c("Baseline (A=1)",
                  paste("Neutral tech: A =", A),
                  paste("Embodied tech: A =", A)),
       col = c("black", "blue", "red"),
       lwd = 2,lty = c(1,2,3))
```

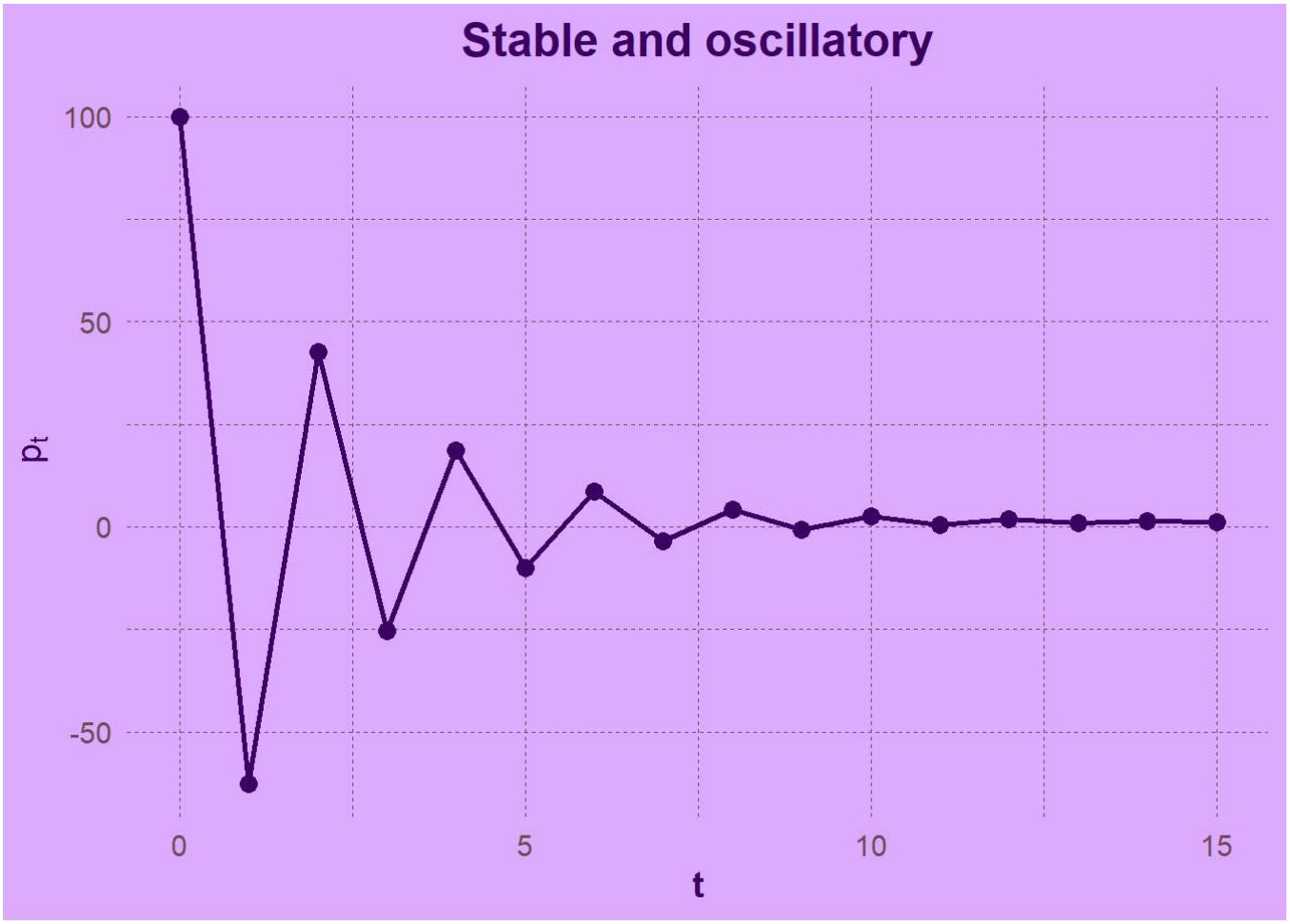

Per-worker Production Function with Technology
Baseline (A=1)
Neutral tech: A = 2
Embodied tech: A = 2
Output per worker (y = Y/L)
0.5
1.0
1.5
2.0
2.5
3.0
3.5
0
10
20
30
40
50
Capital per worker (k = K/L)

```r
#graph 9
# Define utility function
u <- function(c, sigma) {
  if (sigma == 1) {
    return(log(c))        # Special case: sigma = 1
  } else {
    return((c^(1 - sigma) - 1) / (1 - sigma))
  }
}

# Set consumption range
c <- seq(0.1, 10, length.out = 200)

# Different sigma values to test
sigmas <- c(0,0.5,1,2,5)

# Plot utilities
plot(c, u(c, sigmas[1]), type = "l", lwd = 2, col = "blue",
     ylab = "Utility", xlab = "Consumption (c)",
     main = "CRRA Utility for Different Sigma Values")

# Add more curves
cols <- c("blue", "red", "green", "purple", "yellow" )
for (i in 2:length(sigmas)) {
  lines(c, u(c, sigmas[i]), col = cols[i], lwd = 2)
}

legend("topleft", legend = paste("sigma =", sigmas),
       col = cols, lwd = "2", cex=0.7 )
```

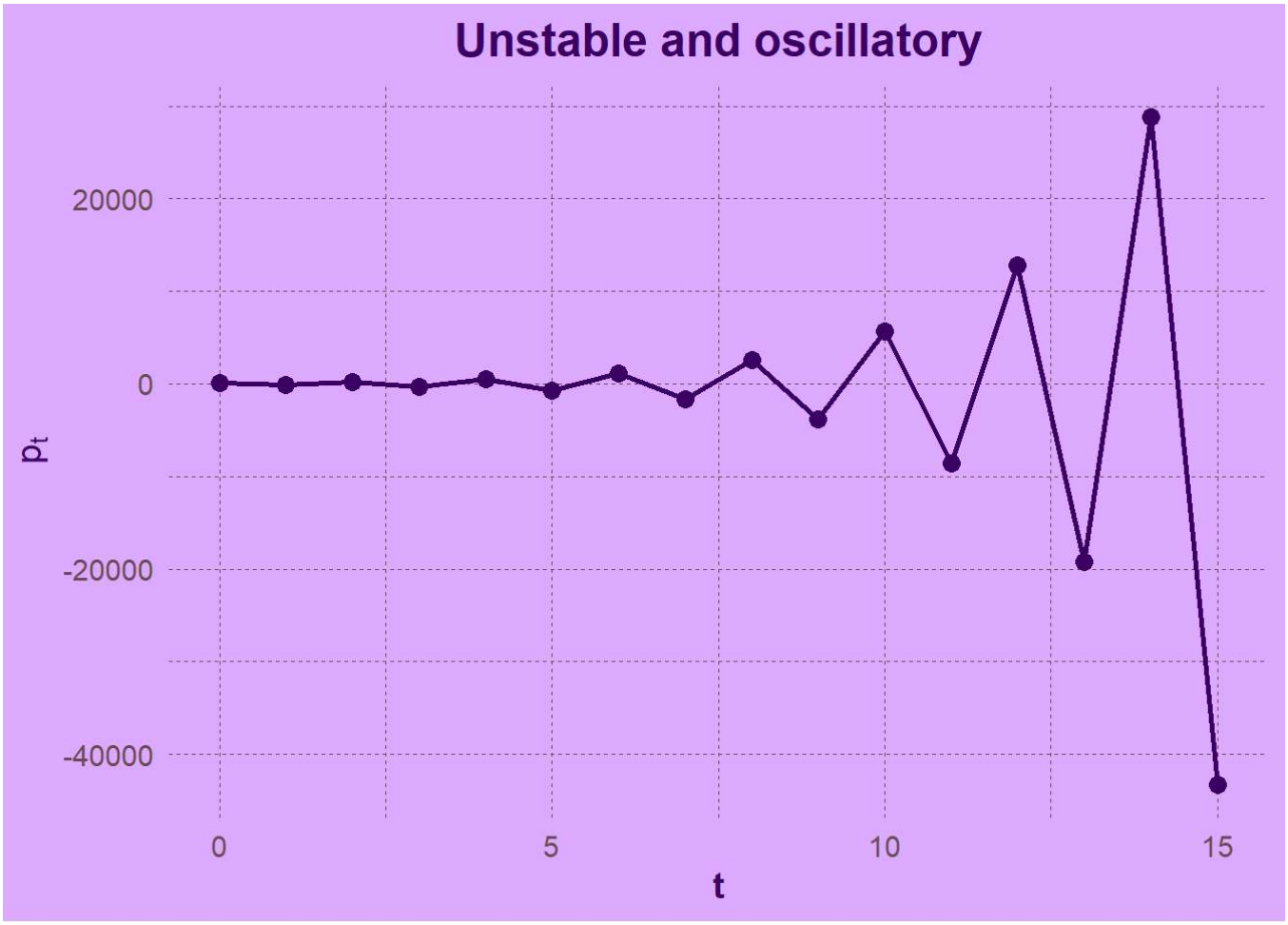

CRRA Utility for Different Sigma Values
sigma = 0
sigma = 0.5
sigma = 1
sigma = 2
sigma = 5
Utility
0
2
4
6
8
0
2
4
6
8
10
Consumption (c)

```r
#question 10

library(ggplot2)

# --- Color palette ---
deeppurple    <- "#3D0066"   # Primary deep purple
lilac         <- "#C77DFF"   # Accent lilac
palelilac     <- "#E0AAFF"   # Soft pale lilac
brightpink    <- "#FF2E92"   # Bright pink highlight
nearblack     <- "#1A1A1A"   # Dark text
mutedbrownish <- "#6D4C56"   # Muted neutral
lavender      <- "#F3E6F8"   # Light background

# --- Simulation parameters ---
d0 <- 4
d1 <- -1.7
s0 <- 0.5
s1 <- 1.1

# Initial condition
p0 <- 100

# Number of iterations
n <- 15

# --- Simulation loop ---
p <- numeric(n + 1)   # Pre-allocate vector
p[1] <- p0

for (t in 1:n) {
  p[t + 1] <- (s1 / d1) * p[t] + (s0 - d0) / d1
}

# --- Prepare data for plotting ---
df <- data.frame(
  t  = 0:n,
  pt = p[1:(n+1)]
)

# --- Visualisation ---
ggplot(df, aes(x = t, y = pt)) +
  geom_line(color = deeppurple, linewidth = 1) +
  geom_point(color = deeppurple, size = 3) +
  labs(
    title = "Stable and oscillatory",
    x = "t",
    y = expression(p[t])
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.background   = element_rect(fill = palelilac, color = NA),
    panel.background  = element_rect(fill = palelilac, color = NA),
    panel.grid        = element_line(color = mutedbrownish, linetype = "dashed", linewidth =
0.3),
    plot.title        = element_text(family = "sans", face = "bold", size = 18,
```

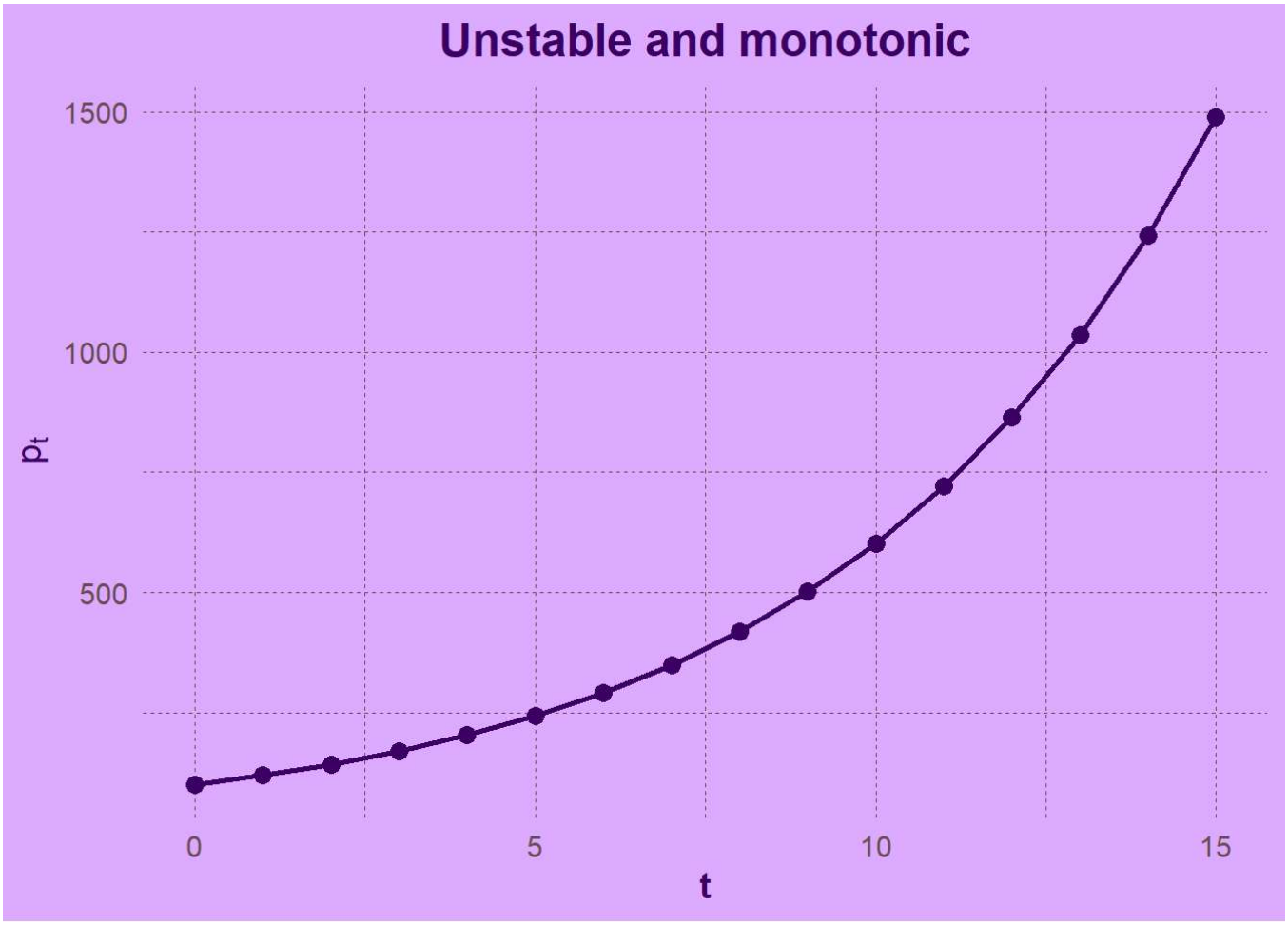
```
                                        hjust = 0.5, color = deeppurple),
    axis.title        = element_text(face = "bold", color = deeppurple),
    axis.text         = element_text(color = mutedbrownish)
  )
```



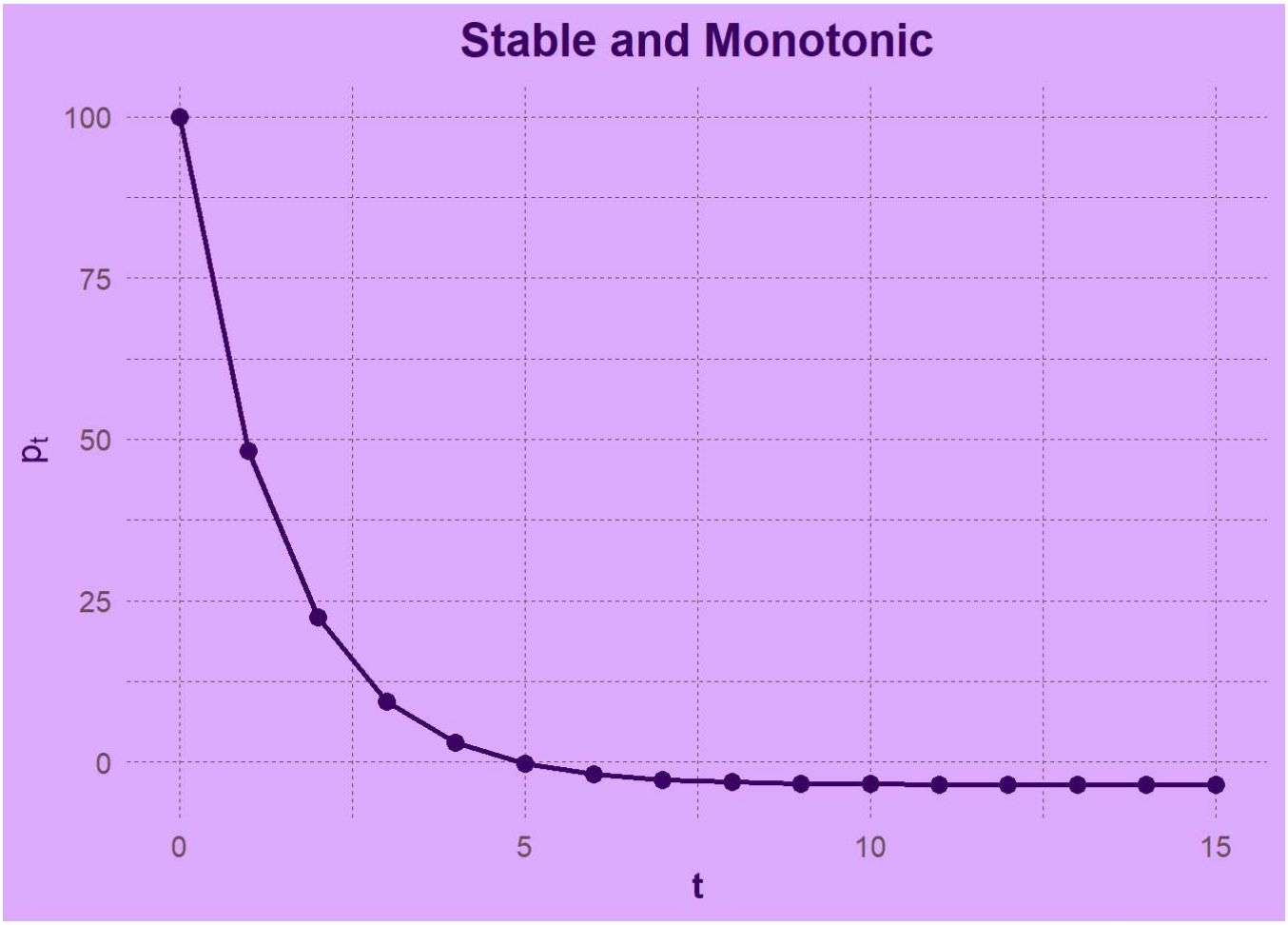
```r
# --- Custom colors ---
deeppurple    <- "#3D0066"
brightpink    <- "#FF2E92"
palelilac     <- "#E0AAFF"
mutedbrownish <- "#6D4C56"

# --- Simulation parameters ---
d0 <- 4
d1 <- -1.2   # Negative slope: standard demand curve
s0 <- 0.5
s1 <- 1.8

# Initial condition
p0 <- 100

# Number of iterations
n <- 15

# --- Simulation loop ---
p <- numeric(n + 1)
p[1] <- p0
for (t in 1:n) {
  p[t + 1] <- (s1 / d1) * p[t] + (s0 - d0) / d1
}

# --- Build dataframe ---
df <- data.frame(
  t  = 0:n,
  pt = p[1:(n + 1)]
)

# --- Visualisation ---
ggplot(df, aes(x = t, y = pt)) +
  geom_line(color = deeppurple, linewidth = 1) +
  geom_point(color = deeppurple, size = 3) +
  labs(
    title = "Unstable and oscillatory",
    x = "t",
    y = expression(p[t])
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.background   = element_rect(fill = palelilac, color = NA),
    panel.background  = element_rect(fill = palelilac, color = NA),
    panel.grid        = element_line(color = mutedbrownish, linetype = "dashed", linewidth =
0.3),
    plot.title        = element_text(family = "sans", face = "bold", size = 18,
                                     hjust = 0.5, color = deeppurple),
    axis.title        = element_text(face = "bold", color = deeppurple),
    axis.text         = element_text(color = mutedbrownish)
  )
```


Unstable and oscillatory
20000
0
-20000
-40000
p_t
0
5
10
15
t

```r
# --- Color palette ---
deeppurple    <- "#3D0066"   # Primary deep purple
lilac         <- "#C77DFF"   # Accent lilac
palelilac     <- "#E0AAFF"   # Soft pale lilac
brightpink    <- "#FF2E92"   # Bright pink highlight
nearblack     <- "#1A1A1A"   # Dark text
mutedbrownish <- "#6D4C56"   # Muted neutral
lavender      <- "#F3E6F8"   # Light background

# --- Simulation parameters ---
d0 <- 4
d1 <- 5   # Positive slope (Giffen goods → monotonic)
s0 <- 0.5
s1 <- 6

# Initial condition
p0 <- 100

# Number of iterations
n <- 15

# --- Simulation loop ---
p <- numeric(n + 1)   # Pre-allocate vector
p[1] <- p0

for (t in 1:n) {
  p[t + 1] <- (s1 / d1) * p[t] + (s0 - d0) / d1
}

# --- Build dataframe for plotting ---
df <- data.frame(
  t  = 0:n,
  pt = p[1:(n+1)]
)

# --- Visualisation ---
ggplot(df, aes(x = t, y = pt)) +
  geom_line(color = deeppurple, linewidth = 1) +
  geom_point(color = deeppurple, size = 3) +
  labs(
    title = "Unstable and monotonic",
    x = "t",
    y = expression(p[t])
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.background   = element_rect(fill = palelilac, color = NA),
    panel.background  = element_rect(fill = palelilac, color = NA),
    panel.grid        = element_line(color = mutedbrownish, linetype = "dashed", linewidth =
0.3),
    plot.title        = element_text(family = "sans", face = "bold", size = 18,
                                     hjust = 0.5, color = deeppurple),
    axis.title        = element_text(face = "bold", color = deeppurple),
    axis.text         = element_text(color = mutedbrownish)
  )
```


Unstable and monotonic
1500
1000
500
p_t
0
5
10
15
t

```r
# --- Simulation parameters for stable monotonic ---
d0 <- 4
d1 <- 2      # positive, ensures monotonic adjustment
s0 <- 0.5
s1 <- 1      # so s1/d1 = 0.5, within (0,1)

# Initial condition
p0 <- 100

# Number of iterations
n <- 15

# --- Simulation loop ---
p <- numeric(n + 1)
p[1] <- p0

for (t in 1:n) {
  p[t + 1] <- (s1 / d1) * p[t] + (s0 - d0) / d1
}

# --- Prepare data ---
df <- data.frame(
  t  = 0:n,
  pt = p[1:(n+1)]
)

# --- Visualisation ---
ggplot(df, aes(x = t, y = pt)) +
  geom_line(color = deeppurple, linewidth = 1) +
  geom_point(color = deeppurple, size = 3) +
  labs(
    title = "Stable and Monotonic",
    x = "t",
    y = expression(p[t])
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.background   = element_rect(fill = palelilac, color = NA),
    panel.background  = element_rect(fill = palelilac, color = NA),
    panel.grid        = element_line(color = mutedbrownish, linetype = "dashed", linewidth =
0.3),
    plot.title        = element_text(family = "sans", face = "bold", size = 18,
                                     hjust = 0.5, color = deeppurple),
    axis.title        = element_text(face = "bold", color = deeppurple),
    axis.text         = element_text(color = mutedbrownish)
  )
```


Stable and Monotonic
100
75
50
25
0
p_t
0
5
10
15
t